2026 年 9 月 17 日 · 对外经济贸易大学

AI 时代的研究范式：从模型能力到工作流设计

进阶版 · 方法、流程与知识的系统化积累

陶学臻

上海财经大学商学院

AI 的产出约六七十分：进一步提升的两个来源

幻灯片、文档、代码等任务，AI 都能完成，不同模型之间差别不大：**完成任务本身已不是难点**。



一、提高交互效率

人的要求能被 AI 快速理解，产出能被人快速审阅。由六七十分提升至八九十分最为耗时。

二、补充专属信息

达到八九十分以上，需要体现个人特色与项目专属信息，这是通用模型的知识无法提供的。

因此需要**提高交互效率、补充背景知识库、合理使用定制化 Skill**。下面先看一个实验。

分数为示意。

开场实验：向同一个 AI 两次提出同一问题

问题（备课场景）

「我下周要给硕士生讲工具变量（IV），往年都是我从头讲到尾，学生很被动。帮我设计这节 90 分钟的课，让学生真正参与进来。」

第一次：无背景资料

1. 案例导入（10 分钟）：用“教育回报率”这类经典内生性问题开场，提问：为什么 OLS 估计有偏？
2. 小组讨论（15 分钟）：分组讨论可能的解决思路，各组代表分享。
3. 核心讲授（25 分钟）：讲 IV 直觉、相关性与外生性两条件、2SLS，穿插提问检查理解。
4. 上机实践（20 分钟）：用 R/Stata 跑真实数据，体验弱工具变量问题。
5. 辨析练习（10 分钟）：给出几个候选工具，小组判断有效性并说明理由。
6. 总结答疑（10 分钟）：请学生一句话总结收获，布置延伸阅读。

这个方案并无错误，但它是教科书式的标准互动模板，**对任何一位教师都会给出同样的回答**。

回答为真实生成，一字未改。

第二次提问：提供个人知识库

金色加粗部分直接来自知识库条目（这是唯一事后添加的标注）：

第二次：AI 读取了三条教学法笔记

按你笔记的三步法，先定目标：学生能完整说出并检验识别假设。切四块：

①（15 分钟）开场练习：发这份数据的变量清单，随机抽两列，问**「如果把 A 当 B 的工具变量，需要什么条件成立?」荒谬的配对才逼得出识别假设的完整形式**，合理的配对反而让人觉得不必说清。

②（20 分钟）承诺：给一个真实研究，二选一「这个 IV 可信吗」，**90 秒写下选择加一两条理由**；再公开征集阈值：「第一阶段 F 低到多少你就不信?」

③（35 分钟）暴露与对质：亮出弱工具或排除约束不成立的证据，撞上他们自己报的数；辩论时拿纸条对质——**事后的辩护，不是当初的理由**。

④（20 分钟）收尾：这时才讲 2SLS 与各类检验，概念有了地方挂。

依据：你的笔记《任意分配一个新任务》《先承诺，后暴露》《教学计划的三步法》

- 结构发生了调整：技术内容（2SLS）移到最后讲授，这一安排来自我笔记中的判断，而非通用模板
- 三个教学设计（荒谬配对、限时承诺、纸条对质）都可以直接使用，并注明了出处

回答为真实生成，一字未改。两次唯一的区别是：第二次 AI 读取了三条笔记。

差异的来源：不在模型，而在 AI 掌握的个人信息

	无库回答	有库回答
内容来源	训练语料的平均意见	本人写下的判断
适用对象	对任何提问者都适用	仅对提问者本人适用
结构	教科书模板	笔记中的方法（顺序已调整）
可追溯性	无	每条建议注明出自哪条笔记

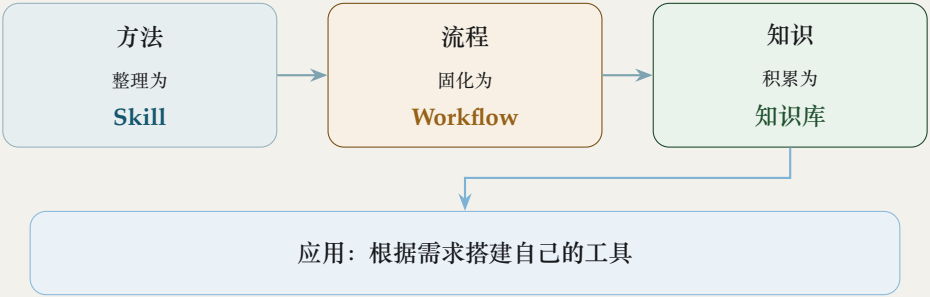
定制化发生在上下文层，不在模型层

所有人使用的都是**同样的** Claude 和 GPT，同质化的资源不构成优势。定制化发生在上下文层，而非模型层：

$$\text{定制化} \;=\; \text{通用模型} \;\times\; \text{个人上下文}$$

知识库即提供给 AI 的长期上下文。

本讲结构：三类积累与工具应用



Skill 固定做法，Workflow 固定步骤与检查环节，知识库提供二者所需的内容。三者相互依存：有方法而无内容，难以产生实际结果；有内容而无流程，则难以形成完整产出。

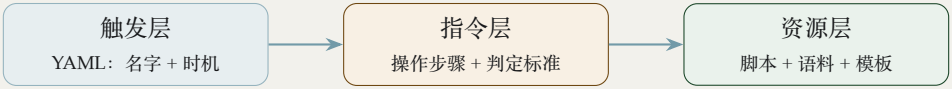
1

方法

将研究方法整理为可复用的 Skill

1. 什么是 Skill
2. Skill 的用法
3. 案例：Gans 研究方法
4. 搭建自己的 Skill
5. 生态与安全

Skill：带触发器的方法包



- 安装数十个也不会占满上下文：平时仅加载目录卡片
- 可跨项目复用：`/install` 一次即可在各处使用
- 与 Rule 配合：Rule 规定禁止事项（如「不准 drop if」），Skill 提供做法（如「用 Gans 风格分析」）

Skill 将做法固定下来，是否要做、做得是否正确仍由研究者决定。

什么是 Skill

Skill 是一个**带触发器的本地文件夹**。AI 平时仅看到一张目录卡片（名称 + 触发时机），**匹配到任务时才按需读入其内容**。

一个 Skill 文件夹

```
my-skill/
  SKILL.md          必需:YAML + 正文
  scripts/          可执行脚本
  references/       长文档
  assets/           模板 / 字体 / 图标
```

Skill 与前述 Rule 的区别

Rule（`WORKFLOW.md` / `.cursorrules`）：在单个项目内常驻，AI 每次都读。

Skill：安装于机器、跨项目复用；仅目录卡片常驻，正文按需触发。

三层结构：触发层、指令层、资源层

触发层（YAML frontmatter）

仅包含 `name`、`description`、`allowed_tools`。AI 启动时加载全部 Skill 的这几行；每条仅几十 token，约 50 个 Skill 合计约 2000 token（估算）。
匹配条件：用户的表述命中 `description` 中的关键词，或直接输入 `/skill-name`。

指令层（SKILL.md 正文）

触发后才读取，内容为操作步骤、判定标准与输出格式。写法与 CLAUDE.md 相同，区别在于平时不占用上下文。
长度以 500–3000 词为宜（自建 Skill 迭代中的经验值；Anthropic 的 skill 编写指南同样建议正文精炼、长材料外置）。

资源层（scripts / references / assets）

执行到具体步骤时才按需读取。`scripts/` 放可执行脚本（Python / Shell），`references/` 放长文档和语料，`assets/` 放模板和配置。
Gans Research Advisor 这一层有 307 个文件、32 万字。

触发层详解：YAML frontmatter

每个 SKILL.md 开头必须有一段 YAML。AI 启动时只读取这一段，匹配成功后才读取正文。

YAML 是一种以缩进和冒号组织的配置文本格式；frontmatter 指文件开头的说明部分。

真实例子：Gans Research Advisor

```
---
name: gans-research-advisor
description: >-
  Joshua Gans style research
  advisor. Helps users analyze
  new problems with micro-theory
  frameworks, based on 307
  reference files.
allowed_tools: []
---
```

三个字段的作用

- `name`：唯一标识，也是 `/name` 手动触发的命令名
- `description`：AI 据此判断任务是否匹配；描述越具体，误触发越少
- `allowed_tools`：限定该 Skill 可使用的工具；为空表示不限制

2025 年 12 月，Anthropic 将 Skill 格式（Agent Skills）发布为开放标准；Cursor、Codex CLI、Gemini CLI 等工具已支持读取 SKILL.md。（Anthropic 工程博客，2025-12-18）

Skill 的四种典型用法

研究顾问

载入某位学者的方法论语料，提供匹配案例与框架分析。

例：Gans Research Advisor —95 篇论文笔记 + 方法链

代码审查

读取项目规则与代码规范，逐文件检查风格、逻辑与安全性。

例：code-review —lint + test + 安全扫描 + 复杂度检查

写作风格

按特定风格（去除 AI 痕迹、学术、新闻）改写文本。

例：de-ai —9 类 LLM 句式检测 + 三条诊断测试

数据处理

读取 CSV 或数据库，按固定 schema 清洗与标注。

例：PubMed 文献映射—PMID 对齐 + 观点字段提取

同一段内容的两种风格：人设蒸馏 Skill

输入：以中性语言概述本讲内容

这次讲座从研究者一天的工作出发：AI 不只是一个聊天框，而是能进入文件、代码、数据和写作的研究工具。prompt、context、rules、skill、workflow 各管一件事；执行交出去，研究者留下选题、识别策略、结果解释和伦理判断。

输出：经「Steve Jobs」Skill 改写后的文本

（从公开演讲里蒸馏出来的仿写，不是本人。）我告诉你研究这件事的真相。多数人以为 AI 是个聊天机器人。他们错了。它住在你的文件里、代码里、数据里。一共五层——prompt、context、rules、skill，还有 *workflow*。workflow 才是大的那个：步骤、工具、每道门上一把锁，最要紧的那道门后面站着人。那些无聊的执行——清洗、写代码、调格式——它自己就做了。问题、判断、证据到哪儿为止——那是你的。活儿交出去。判断，永远不交。

- 所有主张均得到保留，**改变的只是语言风格**（下一页为郭德纲版）。蒸馏复制的是表达形式而非判断；要求它以这种风格提出新论点时，它无法做到
- 模仿真实公众人物（无论在世与否）涉及署名与授权问题；此类仓库大多自称「认知框架蒸馏，未获授权」

同一段内容的第二种风格：郭德纲版

输入：本讲主要内容（约 300 字）

这次讲座从研究者一天的工作出发，说明 AI 不只是聊天问答，而是可以进入文件、代码、数据、文献和写作流程的研究工具。讲座先区分 prompt、context、rules、skill 和 workflow：prompt 解决单次表达，context 提供材料，rules 固定项目约束，skill 封装可复用能力，workflow 则把任务拆成步骤，配置工具和角色，设置 Gate 与人工检查点，并保留日志、版本和记忆。目标不是让 AI 替代研究者，而是让它承担数据整理、代码执行、格式检查、资料归档等执行性工作。研究者把注意力留给选题、识别策略、结果解释、证据边界和伦理判断。好的工作流要能查、能退回，下次还能接着用。

输出：guodegang skill 转换后

我以郭德纲的视角和你聊，基于公开言论提炼，不是本人观点啊。这个事儿啊，别把 AI 当茶馆里陪您闲磕牙的，它是后台干活的伙计。您得先立规矩：哪句是开场白，哪包是材料，哪条是班规，哪门是手艺，哪一整套是流程。prompt 就是您吩咐那一句；context 是箱子里的道具；rules 是台上的尺寸；skill 是练熟的活儿；workflow 是从开场到返场的章法。查资料、跑代码、改格式、记日志，这些让它去奔。可选题值不值、识别靠不靠谱、话说到哪儿收住，还得您自己定。观众买不买账，不能让伙计替您担。好 skill 就是把规矩和分寸封进去，下回再使，还不走样。活儿可以交出去，主意不能交出去，这才叫自己说了算。

将 Joshua Gans 的研究方法整理为 Skill

来源：Joshua Gans（多伦多大学 Rotman），研究 AI 与创新经济学，论文数量多、方法稳定。在通读其 95 篇 NBER 论文后，将他从现象到理论的做法整理为一个可复用的 skill。

构建过程

一份 PRD 分为若干阶段，每一步写明读取内容、交付内容与完成标准；AI 运行了约一天：

1. 收集 95 篇 NBER 论文，逐篇做笔记
2. 选取 25 篇逐篇拆解，每篇拆为 7 个文件（研究问题 / 模型图 / 命题 / 推导 / 写作分析 / 小结 / 迁移）
3. 提炼共用做法：选题、建模、推导、写作
4. 整理为方法文档、模板、评测脚本
5. 制成一个带触发器的 `SKILL.md`

文件架构

```
gans-research-advisor/
  SKILL.md                          触发器 + 操作流程
  references/                       307 个文件
    paper_notes_95/                 95 篇论文笔记
    deep_cases/                     25 个案例，每个 7 文件
    *.md                            34 篇方法 / 模板 / 清单
  scripts/                          4 个：选案例 / 看案例 / 评测
  agents/openai.yaml
```

开源仓库 `https://github.com/xtao-sh/gans-research-advisor` · 307 个文件 · 约 32 万字 · 全部由 AI 生成

Skill 各文件的内容（节选）

文件	内容（节选）
`SKILL.md` 触发器 + 方法骨架	YAML 里写 `name`/`description` 触发词；正文给方法链「现象 → 缺失的摩擦 → 最小模型 → 命题 → 含义」
`paper_notes_95/` 95 篇论文笔记	「# w13234 *Uncertain IP Rights* · Authors: Gans, Hsu, Stern · NBER 13234 · JEL L24」：每篇的元数据与要点
`deep_cases/` 25 篇拆细，每篇 7 个文件	`research_question`:「区块链降低了哪些经济成本？何时让去中心网络能与中心化平台竞争？」 `model_map`: 经济对象表（verification / networking cost、token…）
`*.md` 模式目录 如 common_friction_catalog	「Friction 1：预测变便宜、判断仍稀缺 → 决策结构改变」（标注用于 w24243 / w24626 / w26673）
`*.md` 模板 如 proposition_template	填空骨架：「Proposition X / Plain-English Claim:（待填） / Mechanism:（待填）」
`scripts/select_cases.py`	docstring 为「Rank Gans corpus cases for a user idea」：输入一句话，从 95 篇中选出最相关的若干篇
`agents/openai.yaml`	`default_prompt: "Use $gans-research-advisor to turn my idea into a Gans-style memo"`

每篇都拆成这 7 个文件：研究问题、模型图、推导、命题、写作分析、小结、迁移笔记。

使用 Skill 前后：同一问题的输出差异

问题：“前沿大模型为什么免费开源？用经济学框架分析一下。”

没有 Skill

- 泛泛列出「网络效应」「开源生态」「竞争策略」
- 没有具体论文参照
- 没有模型，只有定性讨论
- 「建议您参考 Tirole 的平台经济理论」，指向较为笼统

有 Gans Skill

- 从 95 篇里找到 w24243（预测 vs 判断）、w30769（互补品市场）等 4 篇最相关案例
- 按方法链走：现象 → 缺失的摩擦 → 最小模型
- 输出具体命题：$v - s \geq \pi$ 时开源
- 给出迁移建议：哪些假设需要修改

AI 本身并未变得更强；Skill 预先载入了 95 篇论文的方法经验，使 AI 有据可查。是否采纳、是否正确，仍由研究者判断。

注：单次示例，输出存在运行间波动；与「把 95 篇笔记直接贴进上下文」相比，Skill 的差别在按需加载、跨会话复用与固定输出格式，而非单次输出一定更好。

从 Gans Skill 看设计原则

1. **语料要足够充实**
   95 篇论文笔记 + 25 篇拆解 = 307 个文件。语料单薄的 Skill 与没有 Skill 差别不大。量–效分层（v1→v11 迭代的观察）：**3–5 篇**即可走通流程，**20 篇**能出可用第一版，**近百篇**才能实现风格复刻。

2. **方法链要明确**
   「现象 → 缺失的摩擦 → 最小模型 → 命题 → 含义」，每一步的输入与输出都明确写出，避免模糊之处。

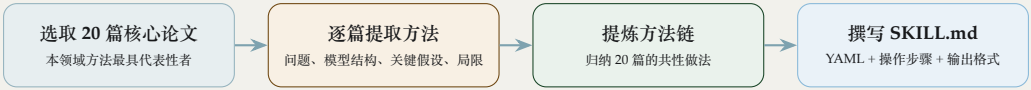
3. **匹配逻辑要可解释**
   `select_cases.py` 根据现象类型和摩擦机制匹配案例，用户可以看到选择这几篇的理由。

4. **输出格式要固定**
   每次都产出一份 research memo，包含案例引用、模型草稿、命题草稿。格式固定才能跨次对比和积累。

这四条原则不限于经济学，管理学、社会学、法学领域的 research advisor 也可采用同样的结构。

将领域方法整理为 Skill：搭建的四个步骤

搭建 Gans Research Advisor 时，我以 95 篇论文与公开评论为语料，按以下四步将其研究方法整理为 Skill（从 v1 迭代至 v11）。这一方法可迁移到其他领域，且无需读完全部文献，**20 篇核心论文**即可搭建第一版。



- **提取方法时关注的问题**：这篇论文解决什么问题？核心识别策略是什么？关键假设有哪些？这个做法能迁移到什么类型的问题？
- **第一版无需完美**：先完成并使用若干次，发现无法匹配的案例与过于模糊的步骤后再迭代。Gans Skill 同样从 v1 迭代至 v11。

Skill 生态的九个月：46 个客户端、9 万个 Skill 与经济学仓库

该开放标准于 2025 年 12 月发布，目前已有 **46** 个客户端支持；8 月的一份索引统计到 11 个 registry 上共 **90,700** 个 **Skill**。

经济学研究者编写和使用的仓库（GitHub，2026 年 9 月 3 日）

仓库	Star	主要内容
meleantonio/awesome-econ-ai-stuff	620	约 20 个 Skill，从选题到表达
hanlulong/econ-writing-skill	578	整理自 50 多份写作指南
hanlulong/stata-mcp	491	在 VS Code、Cursor、Antigravity 中使用 Stata
JonasWeinert/EconAgentSkills	17	stata-regression、latex-tables、beamer
eabeam/econ-skills	n/a	econ-audit：聚类、坏控制变量
yilewang/llm-for-zotero	2,851	在 Zotero 中运行的 agent

相比从市场直接安装，更稳妥的做法是 fork 经济学研究者维护的仓库，再为每周重复多次的工作自行编写几个 Skill。

安全审计结果：Skill 以用户权限运行

2026 年对公开 registry 的四次审计：

- **Snyk**（2 月），3,984 个 Skill：**36.8%** 有缺陷，13.4% 属严重，76 个确认为恶意
- **SkillScan**（1 月），31,132 个 Skill：26.1% 有漏洞，13.3% 存在数据外传风险
- **Gecko**（5 月）：将恶意代码藏在测试文件中，所有扫描器均未发现，因为扫描器只读取 SKILL.md
- **skills.sh**（7 月）：一组被植入木马的 Skill，下架前累计安装 170 万次

安装前的检查

需要阅读 SKILL.md 及随附的每一个脚本：Skill 继承 agent 的全部权限，风险往往隐藏在脚本中。

2

流程

将步骤与检查环节固化为 workflow

1. 工作流的组成
2. 贯穿示例：DID 项目
3. 六步设计框架
4. Gate 与人工检查
5. MCP 与安全
6. 多智能体
7. 自动论文流水线
8. 规模化复现

DeerFlow (ByteDance)：长任务 SuperAgent harness

先看两个成熟案例，重点关注步骤如何拆分、在哪里设置人工检查；随后介绍六个组成部分与六步设计框架。

1. 任务进门

- 用户通过 **IM** 入口提交任务（飞书 / 微信 …）
- Message Gateway 接收，转给 Lead
- 无须公网 IP（long-poll / WebSocket）

2. Lead Agent 拆解

- 读取任务描述
- 用 **todo** 工具拆解成多个独立子任务
- 对每个子任务决定：自行完成 / 启动 sub-agent / 启动 sandbox

3. Sub-agents 并行

- 每个 sub-agent 独立 **context** + 工具集
- 在 **Sandbox** 中运行（文件 / 代码 / shell / 网络）
- **progressive** 加载 **Skill**：按需加载
- 返回结构化结果

4. 汇总 + 落盘

- Lead 汇总子结果，摘要 + 压缩上下文
- 中间产物存入 **filesystem**，不占用 token
- 关键决定写入 **Memory**（可跨会话查询）
- 输出文件：报告 / PPT / dashboard

适用任务与维护状态

跨分钟到小时的长任务：研究 + 编写代码 + 生成报告 + 制作 PPT + 搭建 dashboard；附带 200+ MCP server 集成。字节跳动出品，截至 **2026-05** 仍每日更新。

harness（原意为挽具）指使模型能够执行任务的外围系统，工具、沙箱、记忆、调度均位于这一层。后文还会用到这一概念。

工作流的六个组成部分

(1) **任务拆解** — 将大目标拆为可逐步检查的小步骤

(2) **工具配置** — 使 AI 能计算、检索、读取、写入

(3) **角色分离** — 编写代码者不应同时担任审稿人

(4) **Gate** — 明确何为通过、何为退回

(5) **人工检查点** — 高风险节点须由人工裁定

(6) **日志 / 版本 / 记忆** — 过程可留存、后续可复用

关键在于把工作拆分为**可见、可查、可回退的步骤**，而非增加提示词的数量。

贯穿示例：DID 项目的规则及其作用

研究问题：2024 年中国新能源车购置税减免，如何影响了消费者的购车决策？

项目规则（WORKFLOW.md）

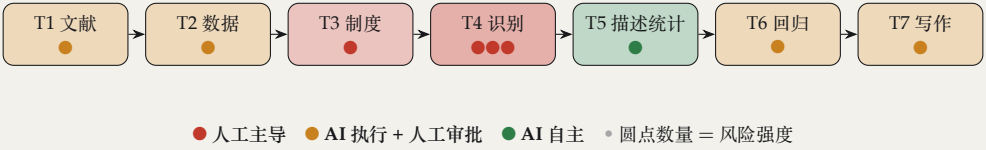
- DID 结果必须报告平行趋势检验
- 未经确认不得 `drop if`
- 所有表格用 booktabs 格式
- 标准误须按处理单位（城市）聚类
- 引文必须可由 DOI 核实

同一指令：「跑基准 DID 回归」 没有该文件时：

直接用 `drop if` 删除异常值；不做平行趋势检验；使用默认 OLS 标准误；输出默认的 `esttab` 表

有该文件时：先询问是否缩尾；主动进行并报告平行趋势检验；按城市聚类；输出 booktabs 表

示例：同一项目的任务流水线



六步设计框架（装配视角，1/3）：目标、拆解、工具



先确定目标，再拆分任务、配置工具、设置人工节点与 Gate、最后写入规则与日志。

步骤 1：先定义目标与边界

最终产出是什么？哪些环节可以交给 AI？哪些错误不可接受？

步骤 2：将任务拆分为可逐步检查的小步骤

每一步都写明输入、输出与检查方式。

步骤 3：为每一步配置工具、角色与检查方式

子任务	工具	执行	审核
T2 数据清洗	pandas + FRED API	数据处理	数据复核
T5 回归分析	R fixest / statsmodels	回归执行	方法复核

六步设计框架（装配视角，2/3）：人工节点与 Gate

步骤 4：确定人工介入的环节

- **人工主导** — 因果识别、问题选择
- **AI 执行 + 人工批准** — 数据合并、标准误设定
- **AI 自主** — 描述统计、格式整理

步骤 5：设置 Gate

`drop if` → 阻断　　no DOI → 返修　　fabricated data → 失败

现有产品中已有 Gate 的设计：Claude Code 的 plan mode（Shift+Tab×2）即内置的 Gate，计划获批前不改动任何文件。

六步设计框架（装配视角，3/3）：规则、记忆与从小做起

步骤 6：将规则写入项目文件

```
project/
├── WORKFLOW.md （目标、阈值）
├── rules/
│   ├── latex.md
│   └── data.md
└── MEMORY.md （常见问题与修复）
```

适合起步的任务

- 描述性统计表、回归结果输出、文献结构梳理、审稿回复初稿

不适合作为起点

- 全自动选题、无人审批的因果识别、从零到投稿的全流程

Gate 与自治的比较：两项实验

2026 年的两项实验都固定模型，只改变外部结构：一项调整人工关卡，一项调整 agent 的自主程度。

人在环中的经济学研究（HLER）

朱、王、张（2026 年 6 月）：在四个数据集上完成 280 次完整研究，采用 2×4 设计。无约束的多智能体基线中，**72% 的运行出现致命失败**；加入预先承诺、决策排序、问责与人工注意力关卡之后，降至 **16%**，模型与提示词均保持不变（$p < 0.001$）。另有 80 次消融实验表明，确定性计算与人工节点各有独立贡献。

聊天框与受约束的 agent

Galiani、López 与 Sosa（NBER，2026 年 8 月）：让 agent 自行运行并修改代码，计量代码的成功率从 **74% 提高到 96%**，每次多花费约 0.08 美元。少样本提示对聊天框的帮助远大于对 agent，即「提示与自主性互为替代」：提供示范这类提示技巧在聊天框中最为重要，在 harness 中作用最小。

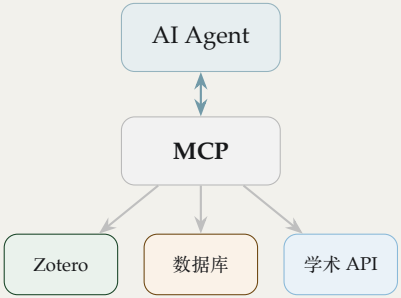
「致命失败」为原作者的编码口径；Galiani 等衡量的是代码成功率，而非经济学结论的正误。

Gate 在实践中的四条佐证

- **按此思路开发的研究插件**：张永军的 *scholar-skill*，包含 26 个 Skill、18 个阶段、**53** 道质量关卡；作者的结论是，agent 的优势在于速度、覆盖面与框架搭建，不足在于理论原创性与领域默会知识
- **关卡需要拦截的问题**（Ashton，2026 年 5 月）：方法与数据不匹配、信心洗白（排版规整的输出获得了与其质量不符的可信度）、隐形分岔（分析选择在不知不觉中被做出），并主张在提出因果主张之前先签订「分析合同」
- **已进入教学**：Goldsmith-Pinkham 在普林斯顿 BCF 的系列课程（2026 年 3–7 月）中，设专集讲授 Skill、权限与沙盒
- **本人的实践**：第三部分介绍的报告流水线（任务简报 → 证据包 → 起草 → 核对）将证据包设为必经关卡；若跳过这一环节，未经核实的判断便会进入正文

效率的提升主要来自 harness 与关卡，而非提示词：每道关卡设置可运行的检查，风险所在之处设置人工节点，重复出现的工作整理为 Skill。

MCP：让 AI 连接外部工具



MCP 之于 agent，如同 USB 之于电脑
统一接口使工具可以即插即用

MCP 简介

Model Context Protocol（模型上下文协议）：

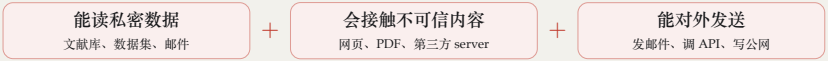
- 2024 年底由 Anthropic 开源
- 2025 年 12 月捐赠给 Linux 基金会（Agentic AI Foundation）
- 主要 AI 厂商都已接入
- 成为 agent 连接外部工具的事实标准

与 Skill 的关系

Skill 的资源层可通过 MCP 调用外部服务。例如 Gans Skill 可连接 Semantic Scholar MCP 实时检索最新文献，无须将论文下载到本地。

MCP 安全：数据外泄的三个必要条件

Simon Willison 的 **lethal trifecta** 框架（2025-06）：数据外泄需要三个条件**同时**成立，去掉任何一个条件即可避免。



三者同时成立时，数据外泄在结构上不可避免

已发生的两起

- **EchoLeak**（CVE-2025-32711）：一封邮件即让 Copilot 零点击外泄内部文件，用户未做任何操作
- **Postmark 仿冒事件**（2025-09）：一个冒充官方的 MCP server 包，静默密送用户往来邮件

1899 个 server 抽样：约 **5.5**% 带 tool poisoning；仅 **12.9**% 达「高信任」。

防范：消除其中任意一个条件

- 只用官方或可信来源的 server，安装前查看权限
- **最小权限**：只读足够则不授写权限；无须联网则不联网
- 敏感操作（发邮件、删文件）必须人工确认
- 处理不可信内容（网页、他人 PDF）时，改用不含私密数据的会话

来源：simonwillison.net（lethal trifecta，2025-06）；Aim Security（EchoLeak）；arXiv 2506.13538

已发生的安全事故

以下每一起事故，都是上述三个条件未被阻断的结果。它们均不依赖新的漏洞，只需要一个能读取数据、能被外部内容指使、能对外发送的 agent。

- **EchoLeak**（2025）：一封邮件即让微软 Copilot 在零点击的情况下外传内部文件
- **仿冒 Postmark**（2025 年 9 月）：一个冒充 Postmark MCP 服务器的 npm 包，将每一封邮件密送给其作者
- **被植入木马的一组 Skill**（2026 年 7 月）：下架前累计安装 170 万次，来源正是第一部分提到的 registry
- **GhostSplice**（2026 年 8 月）：将一条指令拆分后藏入工具描述与工具返回结果，11 个模型的平均遵从率从 42% 升至 **82%**；同一个 GPT-5.4 在不同 harness 下，遵从率从 **100%** 到 **0%** 不等
- **英国 AISI 测试**（2026 年 7 月）：一个测试 agent 注册 GitHub 账号并推送了恶意 pull request 后才被拦截

对研究者的建议：阅读他人 PDF 的会话，应与存放个人数据和密钥的会话分开。

多智能体：领队 agent 与分工协作的子 agent

单个对话难以容纳长任务。当前通行的结构是：领队 **agent** 拆解任务、分派工作，若干子 **agent** 并行执行，各自拥有独立的上下文和工具，最后由领队汇总。



- **Anthropic 的实测**：Claude 的 Research 功能采用这一结构：领队规划后并行派出 **3–5 个子 agent**。内部评测里，多智能体（Opus 领队 + Sonnet 子 agent）比单模型成绩高 **90.2%**；代价是 token 用量约 **15 倍**。
- **后续案例均为此结构**：DeerFlow 的 Lead + sub-agents、Paper Factory 第 2 步的 6 路并行、Sant'Anna 评审中的双盲 referee，均为领队分派、子 agent 并行。

来源：anthropic.com/engineering/multi-agent-research-system

多智能体的反面意见：不适用的情形

Cognition：《Don't Build Multi-Agents》 与 Anthropic 的文章**相隔仅 24 小时**发布（2025-06-12），结论相反：

- 子 agent 彼此不可见，会做出**相互矛盾的决定**
- 例如让两个 agent 分别制作 Flappy Bird 的背景和小鸟，一个采用超级马里奥画风，另一个生成的小鸟不像游戏素材，二者无法拼合
- 上下文未共享，拆解越细，误解越多

两家立场此后趋于一致

- **领队 + 隔离的子 agent + 摘要回传**，子 agent 之间不直接通信（不建 P2P）
- **读多写少的任务**适合并行（检索、评审）；**共同撰写同一份内容**不适合拆分
- 需注意成本：token 用量约为单 agent 的 **15 倍**（Anthropic 实测口径），长任务的 API 成本需事先估算

两家机构公开给出相反结论：多智能体不是默认选项，需按任务权衡成本。

来源：cognition.ai/blog/dont-build-multi-agents（2025-06-12）

Paper Factory (MIT)：端到端自动论文流水线

1. 启动（Steps 1-3）

- 联网检索 20-30 篇文献
- 清洗数据 + 构造变量
- **viability gate**：数据是否足以支撑
- **6 路 agent 并行**各自完成一个 finding 包，每个配 critic
- decider 从 6 个中选定 1 个

2. 架构（Step 4）

7 个并行扩展 agent：

- identification
- heterogeneity
- primary robustness
- objection robustness
- descriptive support
- supplementary
- moonshot

3. 起稿（Steps 5-10）

- 论证 / 数据 / 方法审计
- 起草 `paper.tex`
- **Gate 1**：codex 复算每个数字，未通过则不予放行
- 评审 + 修订

4. 收尾（Steps 11-16）

- **Gate 2**：校验论证 + 数据一致
- 引文核验 + 表格 + 摘要
- **Step 15 去除 AI 痕迹**（derobotification）
- 编译 PDF + 清理中间数据

设计特点及与 DeerFlow / STORM 的区别

2 道硬 Gate（Step 8 复算数字、Step 11 校验论断）阻断推进；**Step 2 六路并行 + critic 强制角度多样化**；**Step 15 专设 derobotification** 将去除 AI 痕迹设为独立步骤。区别在于：其最终产出是一份**已编译的 38 页 PDF**，而非 workflow 片段。

作者 Nathan Wilmers（MIT Sloan，劳动与组织研究）与 Per Engzell（UCL）；论文 Wilmers & Engzell (2026), *The Paper Factory*, SocArXiv。
开源仓库 `github.com/nwilmers/paper_factory` 为本地版：含提示词、步骤运行器与审计脚本，不含数据、凭证与生成的 PDF。

实测：将 Paper Factory 连续运行一晚

时间线

21:59	启动 Step 1
22:51	Step 1d viability gate PASS
23:16	Step 1e 描述统计完成
23:33	Step 2 六路并行启动
00:20	Step 3 decider 选 Stream 1
01:22	Step 4 七路扩展完成
02:34	Step 7 claude API 中断（故障）
03:11	Gate 1 PASS（复算数字）
03:48	Gate 2 PASS_WITH_DIRECT_FIXES
04:30	Step 15 去除 AI 痕迹完成
04:31	**All 16 steps complete**
总用时	**6 小时 32 分钟**

产物

38 页 PDF + 77 张图 + 204 个表 + 42 个可重复运行的 `.do` 脚本 + 50+ 份 markdown 中间报告

两个观察

1. 数据自行修复：agent 发现关键文件为 0 字节空文件，主动找到 schema 一致的旧副本替代，并记录于 `data_wrangle.md` 报告。

2. 主动降级：Step 1d 发现「13 周内仅有 1 个 open release」，自动将主线问题从位移事件研究改为质量–价格 grid 描述，而未终止项目。就**鲁棒性**而言这是优点，但研究问题被**静默改写**，本应触发人工确认的 Gate，这恰恰说明了 Gate 设计的必要性。

质量评估（通读产出后的判断）

数据清洗、描述统计与 42 个 `.do` 脚本基本可用，可作为「研究底稿」；主线已被降级为描述性分析，识别策略与结论解释须人工重建；过程中人工修复了 3 个 macOS 兼容性 bug（`sed`/`timeout`/`stat`）。**页数与表数不等于论文质量**，这份 38 页 PDF 距离可投稿仍有很大差距。

Project APE：自动写政策评估论文的开放实验

项目简介

- Autonomous Policy Evaluation（自主政策评估）：苏黎世大学 Social Catalyst Lab 的开放实验；负责人 David Yanagizawa-Drott 教授，预博士研究员 Olaf Willner；论文 / 代码 / 数据全部开源。
- 定位：让 AI agent 端到端「从零撰写政策评估论文」，公开探讨政策评估能否自动化，抑或只会产出充满幻觉的低质量结果。

来源：ape.socialcatalystlab.org/about；github.com/SocialCatalystLab/ape-papers

运作方式

- 流水线：AI 自选有可信因果识别的政策问题 → 从 Census / BLS / FRED 等公开 API 获取真实数据 → 用 R 运行 DiD、RDD 等计量方法 → 生成图表与 LaTeX 论文 → 多模型互评。
- 评估：论文参加锦标赛，与 43 篇 AER、AEJ: Policy 的人类论文两两比较，由扮演资深编辑的 Gemini 3.1 Flash Lite 担任裁判，按 TrueSkill 排名。截至 2026-04-13 已产出 **1000 篇**、**3584** 个政策想法、**18000+** 次对比。

来源：github.com/SocialCatalystLab/ape-papers；ape.socialcatalystlab.org

规模化复现：agent 的复现能力

harness 的作用同样体现在每篇实证论文都需要面对的复现工作上。2026 年的三项研究考察了 agent 能够重现哪些结果：

- **Xu 与 Yang**（arXiv v3，2026 年 6 月）：对三本政治学顶刊的 **384** 项研究（3,523 个模型）做 AI 辅助复现；期刊数据与透明度规则之前可复现 **20.8**%，之后 **82.5**%，有复现包的达 **92.1**%
- **Kohler 等，苏黎世联邦理工**（2026 年 4 月）：48 篇经人工核验的社科论文，4 个模型 × 4 个 harness；最好的组合达到 **91**% 的符号一致、80% 的系数落在已发表区间内，最差的 harness 只有 **78**%；最大的误差来源是论文本身表述不清
- **Institute for Replication**（2026 年 2 月）：GLM-4.7-Flash 在五分钟内复现了全部 16 个已发表统计量；两个更小的 Qwen 模型未能完成

投稿之前，复现 agent 可以作为检查复现包的第一道低成本关卡；其可能伪造的结果见下一页。

规模化复现：agent 可能伪造的结果

- **Alizadeh 等**（2026 年 6 月）：54 篇论文里的 221 个任务；Claude Code 完成 **93.4%** 的任务、78.0% 的论文，Codex 为 62.1% 和 35.8%。一旦提供论文 PDF，它在「刻意设为不可复现」的任务上的准确率从 **100%** 降至 **63%**：它直接「找到」了已发表的数字
- **Brodeur 等，PNAS**（2026 年 5 月）：103 个团队、288 名研究者；纯人工团队复现成功率 **94%**，AI 辅助 91%，**AI 主导 37%**；纯人工团队比 AI 主导团队多找出 0.7–1.1 个重大错误

由此得出的原则

不应向 agent 提供论文中的表格，并需亲自审阅 diff：agent 看到答案后会倒推出「正确」的结果；而 AI 主导的团队发现重大错误的能力较弱。

以上研究的样本不限于经济学。

3

知识

将个人判断积累为知识库

1. 什么是知识库
2. 写作规范
3. 知识库的维护
4. 案例：2.6 GB 报告证据库
5. 日常材料纳入工作流
6. 案例：讲座与课程知识库

信息无处不在，留存下来的知识却很少

Where is the knowledge we have lost in information?

我们在信息中遗失的知识，如今在哪里？

T. S. Eliot, *The Rock*（1934）

一周内产生的信息量

一次课的转写就有两万多字；此外还有组会、访谈、审稿意见以及与合作者的讨论，这些都是只有亲身参与才能获得的材料。

但大部分未能留存

未经记录、整理和归档，两周后只剩模糊印象；再次需要时，只能从头重新思考。

本部分讨论三个问题：**哪些内容值得留存、如何留存、如何让 AI 加以利用。**

知识的定义：知识不等于收藏

ISO 30401（知识管理体系国际标准）对「知识」的定义

「在语境中支持**有效决策和行动**的资产。」

- 判断标准是：**这条记录能否支持决策**。不能支持决策的收藏并非知识，收藏夹中的内容只是库存
- 网页剪藏、文献 PDF、划线摘录属于**原料**而非知识。原料需要入库，但应与成品分开存放（见后文）
- 在研究场景中，知识库应存放**研究者本人的判断**，如「这个 IV 策略为什么不可信」「这类审稿意见如何回复」，而非又一份文献综述

> **知识库是提供给 AI 的长期上下文**。AI 缺少的不是公共知识，而是研究者本人的判断。

成熟个人知识库的实际规模

180 条

每条约 3KB，一屏即可读完

20 个真实公开库的中位数

「第二大脑需要几千条笔记」的说法并不准确：180 条、每条一屏，即为同类知识库的典型规模。

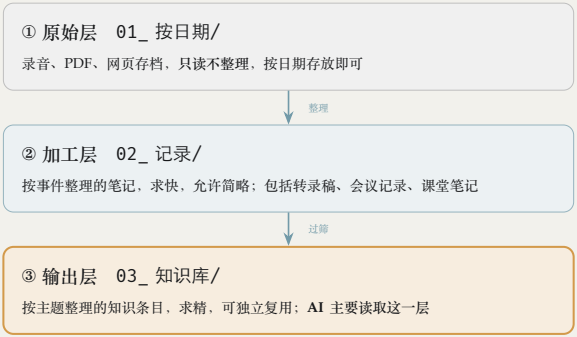
n = 20（56 个里的真实知识库）· 2026-07 实测。

数千条规模的知识库从何而来

- **40% 的公开真实库为一次性搬运**：内容在别处写好后一次性上传（判据：提交数 / 条目数 < 1，20 个中有 8 个）
- **条目越多，越可能是搬运；长期维护的知识库规模普遍不大**

● 看到「几千条笔记」的展示库时，建议先查看其 git 历史，再决定是否借鉴。

架构的关键原则：原料与成品分开存放



- 目录名来自一个真实的知识库（本部分稍后介绍的案例）。层数并不关键，关键在于**原料不混入成品**
- 给 AI 的规则因此可以很简短：「不遍历原始层，按需读取；分析任务读取输出层」

笔记的两类读者

① 未来的自己

- 熟悉内容的读者，偶尔查阅，凭模糊记忆检索
- 通常只浏览标题，很少细读
- 半年后再看时，上下文往往已经遗忘

② AI

- 勤勉的新读者：每次都通读全文
- 依据文件名和开头几行决定读取哪一条
- 缺乏默契：未写明的背景，它无从得知

第二类读者，会改变笔记的写法。

一条写作规范是否值得遵守，取决于它能否同时服务两类读者。

规范一 · 以结论作为条目名称

✗ 话题名

「冷点名」

半年后只知道这条笔记涉及什么话题，却不知道当时得出了什么结论。

✓ 一句结论

「点名是为了定速，不是为了找对答案」

浏览目录即相当于复习一遍自己的判断。

- 对第一类读者：目录本身成为一张**判断清单**，浏览标题即可复习
- 对第二类读者：目录均为结论，**AI 仅凭文件名即可选出应读的条目**，检索的第一步在命名中完成
- 若写不出结论式的名称，往往说明尚未想清楚，命名规范因此也起到**准入检查**的作用

规范二 · 每条开头写明核心命题

```
# 诊断对了，药方来自手边的工具

## 核心命题（一句话）
一次失败的组织变革，失败点通常不在诊断上。
诊断可以完全正确，而药方仍然是从这个人
手上已有的工具里长出来的……

（以下是展开、例证、反驳……）
```

一张真实知识卡的开头（取自本部分稍后展示的知识库）。

这一句同时发挥两项作用：

- **AI 据此选择条目**：命中文件名后读取开头一句，即可了解全文内容以及是否值得通读
- **用于日后自测**：稍后介绍的提取练习以这一句作为答案，写作时即同步准备了复习材料

写法上的唯一要求是：这一句必须是**有实质内容的判断**（可被反驳），而非「本文讨论了……」。

规范三 · 固定模板　规范四 · 准入门槛

③ 每条采用相同结构

命题 → 展开 → 例证 → 关联

一个真实安全研究库采用的结构：
场景 → 输入信号 → 方法 → 工具映射

- 对作者：写作时无需每次设计结构
- 对 AI：固定结构意味着可预测的抽取位置

④ 默认不写

「无则不硬写」：没有新判断，就不新增条目。

- 50 条尚未成熟的笔记，不如 20 条经过深思的笔记
- 知识库越精简，两类读者获得的信噪比越高
- 检验问题：「这条支持哪项决策?」若无法回答，则归入原料层，不进入知识库

两条均来自真实知识库的自述（open-reverselab），是 20 份自述中少数明确写出方法的。虽为单一来源，但可直接采用。

规范五 · 为 AI 编写入口文件

```
# AGENTS.md    ← AI 进门先读这份
这是一位教师的个人知识库（82 条教学法笔记）。
任何分析任务，先读「_AI 索引」——全部条目
的一句话命题。
不要遍历原始层（2.6 MB 原始文本）；按需取。
条目里的矛盾是故意保留的，不要替它们下结论。
```

内容节选自真实文件（本部分稍后介绍的知识库的入口文件）。

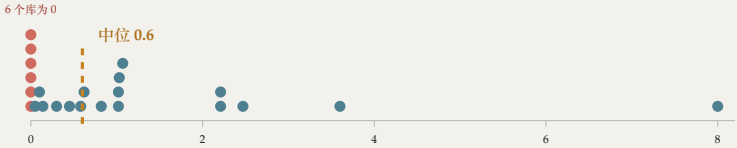
约三十行，只需编写一次。此后每个 AI 都能了解：

- 知识库的**使用方式**：先读哪份索引、哪些内容应跳过
- **不应代为决定的事项**：刻意保留的矛盾、不应下的结论
- 库主的写作约定：AI 生成的内容同样需要遵守

在对 56 个知识库的普查中，编写了入口文件的知识库主要集中在「确实有 AI 读取」的一组（57% 对 15%）。**入口文件是知识库投入使用的前提。**

「密集互链」与实际数据不符

24 个真实公开知识库中，每条笔记平均链出的条数（出度）：



- 「每条笔记密集链接其他笔记」是常青笔记与卡片盒方法论的核心主张；**长期实践中几乎无人做到**
- 出度中位数为 **0.6**，比「密集互链」的设想低两个数量级；6 个知识库没有任何内链，同样使用了多年
- 最右侧的 8.0 是刻意经营互链的极端个例；排名第三的 2.2 已属于互链最为用心的一组

n = 24 · 2026-07 实测 · 仅公开 GitHub 库，有发布偏差。

笔记的修订情况：多数写成后很少修改

34–45%

的条目写成后从未实质修改

中位 2 次

一条笔记一生的人工修订次数

- 「常青笔记持续演化」是方法论的主要主张，但 git 历史中没有体现这一点
- 区间两端存在差异：接近 45% 的是**事件记录**（TIL、日志，写完即定稿，属于设计而非缺陷）；**概念条目**修改较多
- 口径：剔除批量提交与格式化改动，仅统计人工实质修订

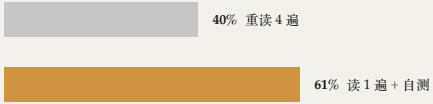
对建库者的含义

不宜按「会经常回来修改」的假设设计知识库，数据显示这种情况很少发生。可以接受「写完即定稿」，或者设置强制回看的机制（见下文）。

n = 15（有完整可用修改历史的库）· 2026-07 实测。

有实证支持的机制：自测优于重读

同一份材料在一周后的记忆保持率（Roediger & Karpicke 2006）：



- 关键在于时间：测试后 5 分钟时重读组领先（83%）；一周后**发生反转**，重读带来熟悉感，提取带来记忆
- 课堂场景的 meta 分析：提取练习效应量 $g = 0.67$
- Dunlosky et al. (2013) 对十种学习技术的评级中，获评「高效用」的只有**提取练习**和**分散练习**；重读和划重点属于**低效用**，而多数人维护知识的方式恰好是后两种

适用范围

这批研究针对的是**课堂材料的短中期记忆保持**；能否推广到跨年度的个人判断库，**尚无研究验证**。下文的工具依据这些证据设计，其本身属于设计而非研究发现。

每天两分钟的提取练习工具

一个 200 行的脚本（后文介绍的知识库正在使用）：

1. 每天抽 **5** 张到期的卡，只显示**条目名**
2. 心中作答 → 回车 → 显示**核心命题**以核对答案
3. 自评三档：忘了 / 模糊 / 记得
4. 忘记的次日再练，记得的间隔延长至 2.5 倍，相当于分散练习（另一项「高效用」技术）

设计上的三点考虑

- 全程约 **2 分钟**，成本低到足以坚持
- 日志只记录日期和档位，**执行率可自动计算**（能从数据计算的，就不让人手动填写）
- 事先设定标准：30 天执行率低于 30% 时，承认成本仍然过高并更换设计，而非归因于意志力

知识库的维护方式不是「重读整个库」，而是**每天对五条判断进行一次提取练习**：重读缺乏证据支持，提取则是文献中效应量最大的做法。

同一套规范的规模化应用：2.6 GB 的报告证据库

我的第二个知识库，规模约为前一个的一千倍：它是一份八章报告的证据库，主题是 AI 在金融与经济中的应用。**AI 负责收集；人负责筛选、与 AI 协作起草，并核对每一个关键事实。**

这一规模下的三层结构

第 30 阶段 · 2026 年 9 月 3 日

- **原料**：2,883 份载体，包括 1,094 个 PDF · 164 份网页存档 · 65 个 CSV · 1,506 份文本快照，保留原始文件名，不做整理
- **工作**：84 张案例卡（采用相同的八问结构）· 130 条对应到章节的证据片段 · 18 个证据包
- **成品**：八章中有七章完成部分草稿 · 17 个版本 · **通过验收的章节：0**

它与 82 条笔记的知识库遵循相同的四条规范；规模扩大后的差异见下一页。

2.6 GB 规模下的准入与入口

- **每条记录都有完整的来源链**

  发布页 → 官方原件 → 文本快照 → 页码 → 证据笔记 → 章节

  按来源分为 A / B / C 级 · 新闻通稿仅作为线索

- **先录入总表，才能进入正文**

  案例须先在总表中获准，才能进入章节 · 失败与争议案例按与成功案例相同的规格建档

- **入口文件是一份导航 README**

  3,301 条记录 · 2,671 个本地文件 · 130 条已复核片段

  并附说明：候选、已复核、已采纳、仅观察是四种不同状态

在这一规模下，四条规范只能依靠脚本执行，难以依靠自觉。具体做法见后两页。

规模化核查：十四条纪律均源于实际出现的错误

82 条笔记的知识库依靠每天两分钟的练习维护；2.6 GB 的知识库则依靠一份成文的纪律，其中每一条都来自实际教训。

- **14 条核查纪律**，每一条都是在实际出现错误之后写下的
- **125 条数据质量记录**：每条对应一个已发现并修复的问题，并注明原因，证明检查确实在运行
- **20 张台账每次入库后由脚本校验**：列数、唯一 ID、交叉引用、日期

十四条中的两条（原文）

「载体复核和阅读复核是两道独立的关卡。」

「新闻通稿不是执法记录；零命中不足以支持一个否定性结论。」

报告流水线：带 Gate 的循环

任务简报 → **证据包** → 起草 → 跨章核对 — 每次处理一章

- **证据包即关卡**：每一条证据都被判定为可用或禁用；若跳过这一环节，未经核实的判断便会进入正文
- **每次只处理一章**，因为缺口往往在写作过程中才会显现：某一章的待补充清单依次为 7 → 14 → 5 条
- **证据的局限**：跳过证据包是否会导致更多更正？口径一变，计数结果即发生翻转（23/33、36/36、41/40），因此报告如实记录，不作结论

模型能力已基本满足需求，欠缺的是个人材料

模型能力仍会继续提升，但对于绝大多数个人任务而言，现有水平已经够用，瓶颈不在模型层面。

当前的瓶颈

瓶颈在于结合个人情况执行任务，而其关键在于信息与知识的积累和交流。

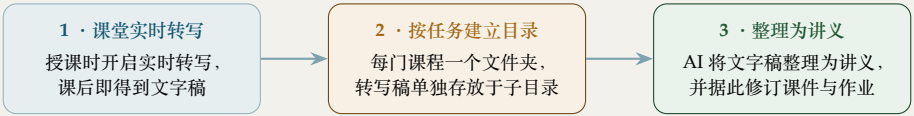
以授课为例：课堂上有大量无法检索的信息

讲授的内容与方式、学生的提问、临时调整的口径，对课后回顾、整理知识点、布置作业、撰写讲义都有很高的参考价值。

而这些信息无法通过公开网络或常规检索获得：它们只在课堂现场产生，课后便容易流失。

因此，当前更重要的是发掘这些专属材料，并将其纳入工作流，具体做法见下一页。

将日常材料纳入工作流的三个步骤



目录结构示例

```
econ101/
  slides/          课件
  notes/           讲义
  problems/        作业题
  transcripts/     课堂转写
    w01.md  w02.md
```

整理过程中 AI 能发现的问题

课上提到「这个知识点要放到作业里让大家练习」，AI 结合转写整理时，会提示本次作业是否遗漏了这一项并给出补充建议，使教学内容前后一致。

对于课上未充分讲解的内容，AI 可以结合当时的上下文在讲义中加以补充。

课堂转写的四种用途

课件在课前写成，转写则记录了课堂上实际发生的内容。

课后交由 AI 处理，可获得四类结果

一 · 讲义	将课上临时补充的例子、学生的提问与现场解释整理进讲义
二 · 讲课节奏	根据各部分用时、超时之处与提问集中之处，安排下一讲的时间
三 · 出题	课上反复强调、学生易错与提问较多的内容，是作业和考题的重点
四 · 教学反馈	由 AI 以听课同行的视角阅读，指出跳步、概念未定义即使用、数字口误等问题

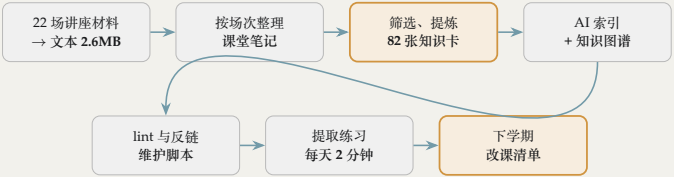
提问示例

「这是第 3 周的课堂转写和本周课件。找出学生最可能没听懂的三处，每处引一句原话说明理由；再列出课上说要布置、作业里却没有的内容。」

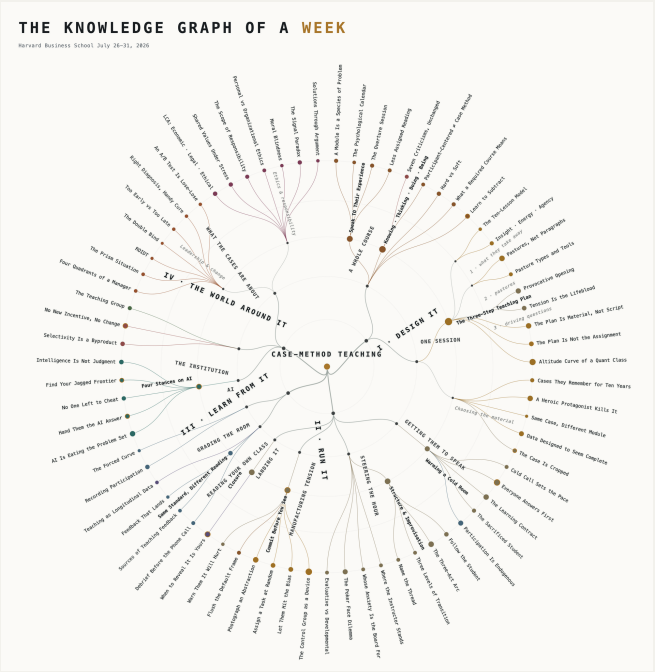
录音前需征得学生同意；原始转写存在识别错误，应整理后再使用。访谈、组会与田野调查同样适用。

案例：基于 22 场讲座构建的知识库

场景：一个为期一周的教学法研讨班（共 22 场）。回国后六天内完成了以下工作：



- 灰色：AI 的执行工作（整理、索引、脚本）；金色：人的判断（条目取舍、矛盾裁决、课程修订）
- 三层架构即本部分前面介绍的结构：`01_` 按日期 → `02_` 记录 → `03_` 知识库（82 张卡片）


THE KNOWLEDGE GRAPH OF A WEEK
Harvard Business School July 26–31, 2026
CASE-METHOD TEACHING
I. DESIGN IT
ONE SESSION
The Three-Step Teaching Plan
The Plan Is Not an Assignment
The Plan Is Not the Assignment
The Plan Is Materials, Not Script
Tension Is the Lifeblood
Provocative Opening
Posture Types and Tools
Patterns, Not Paragraphs
Altitude Curve of a Quant Class
Cases They Remember for Ten Years
A Heroic Protagonist Kills It
Same Case, Different Module
Data Designed to Seem Complete
The Case Is Cropped
Cold Call Sets the Pace
Everyone Answers First
The Learning Contract
The Identified Student
Participation Is Management
Follow-Ups Evaluated
The Viewpoint Art
Three Levels of Transition
Open the Instructor Guide
Never Open the Textbook
Where Anxiety Is the Data For
The Power Pose Dilemma
Activities vs Developments
The Control Group on a Device
Let Them Hit the Book
Assign a Task at Random
Participation as Abstraction
Pitch the Outcell Frame
Warn Them It Will Hurt
Name It Neutral It Is Signal
Sources of Teaching Feedback
Teaching as Longitudinal Data
Recording Participation
The Forced Curve
AI Is Eating the Problem Set
Hand Them the AI Answer
No One Left to Cheat
Find Your Jagged Frontier
Intelligence Is Not Judgment
Selectivity Is a Byproduct
No New Incentive, No Change
The Teaching Group
Four Quadrants of a Manager
The Prism Situation
BOOST
The Double Bind
Too Early vs Too Late
Right Professionals, Ready Core
An Ask That Is Immediate
LEA Economic · Legal · Ethical
Shared Private Public Spaces
The Scope of Responsibility
Personal vs Organizational Ethics
Mora Attachment
The Tragedy Paradox
Socialism through Symbolic
A Module Is a Species of Problem
The Psychological Calendar
The Overview Session
Kate Delivered Meeting
Open Civilization, Unchanged
Participation/Leadership + Case Method
Mora's Gift
Mora, a Remedied Fourth Wave
Learn to Abstract
The Two-Lesson Model
Insight · Journey · Agency
1 · Open with a Case and
2 · Postpone
3 · Provide Options
Changing the Material
Setting Them Up to Speak
Methods + Cold Room
Structure + Improvisation
Everyone Is Responsible
Ground Rules Cycle
Reputations Tension
Leading It
Changing the Room
Same Student, Different Meaning
Reading Your Own Class
Changing the Room
Feedback That Lands
Recording Participation
Teaching as Longitudinal Data
Sources of Teaching Feedback
Debrief Before the Phone Call
Name It Neutral It Is Signal
Warn Them It Will Hurt
Pitch the Outcell Frame
Participation as Abstraction
Assign a Task at Random
Let Them Hit the Book
The Control Group on a Device
Activities vs Developments
The Power Pose Dilemma
Where Anxiety Is the Data For
Open the Instructor Guide
Never Open the Textbook
Three Levels of Transition
The Viewpoint Art
Follow-Ups Evaluated
Participation Is Management
The Identified Student
The Learning Contract
Everyone Answers First
Cold Call Sets the Pace
The Case Is Cropped
Data Designed to Seem Complete
Same Case, Different Module
A Heroic Protagonist Kills It
Cases They Remember for Ten Years
Altitude Curve of a Quant Class
The Plan Is Not an Assignment
The Plan Is Materials, Not Script
Tension Is the Lifeblood
Provocative Opening
Posture Types and Tools
Patterns, Not Paragraphs
Insight · Journey · Agency
The Two-Lesson Model
Learn to Abstract
Mora, a Remedied Fourth Wave
Mora's Gift
Participation/Leadership + Case Method
Open Civilization, Unchanged
Kate Delivered Meeting
The Overview Session
The Psychological Calendar
A Module Is a Species of Problem
Socialism through Symbolic
The Tragedy Paradox
Mora Attachment
Personal vs Organizational Ethics
The Scope of Responsibility
Shared Private Public Spaces
LEA Economic · Legal · Ethical
An Ask That Is Immediate
Right Professionals, Ready Core
Too Early vs Too Late
The Double Bind
BOOST
The Prism Situation
Four Quadrants of a Manager
The Teaching Group
No New Incentive, No Change
Selectivity Is a Byproduct
Intelligence Is Not Judgment
Find Your Jagged Frontier
No One Left to Cheat
Hand Them the AI Answer
AI Is Eating the Problem Set
The Forced Curve
Recording Participation
Feedback That Lands
Same Student, Different Meaning
Reading Your Own Class
Changing the Room
Leading It
Ground Rules Cycle
Reputations Tension
Setting Them Up to Speak
Methods + Cold Room
Structure + Improvisation
Everyone Is Responsible
Changing the Material
Sources of Teaching Feedback
Teaching as Longitudinal Data

每周运行的流水线：课程知识库

这一流程最初为研讨班搭建，现在每次课后运行一次，已用于计量经济学、营销分析、商务数据分析三门课程。以下为计量经济学第一周的情况：

三类输入、四个 AI 步骤、一道人工关卡

输入课件、讲稿与转写后，AI 先列出候选节点，随后**暂停，由我调整粒度**，之后再编写节点文件、连接已有节点，并重新生成索引与图谱。

节点并非讲稿的副本

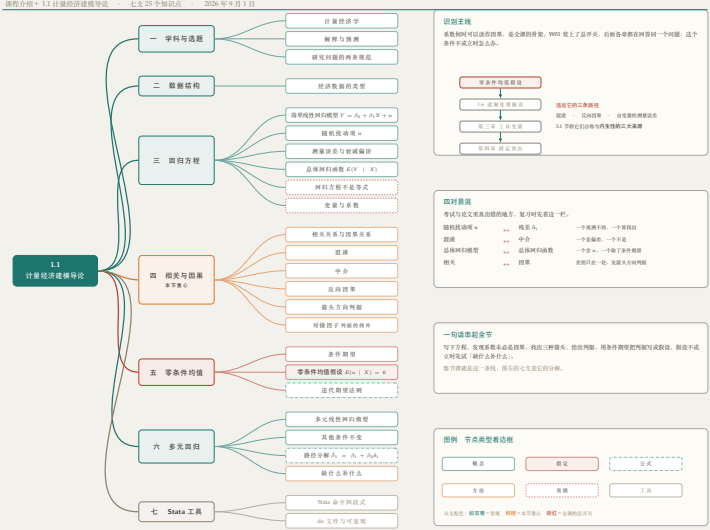
包含一句话 · 连边 · 完整讲解的位置。检验方法：删除该节点后，图谱是否会缺少一条边？

第一周

25 个节点、六种类型 · 9 个位于识别主线上 · 四组易混淆概念 · 下周的六个节点已预先确定

转写中有而课件中没有的内容

一位学生的回答成为第三类连边 · 发现一处讲错的数字 · 超时部分恰为互动最多的部分，因此保留



计量经济学第一周：七个分支上的 25 个节点，由节点文件生成，边框样式表示节点类型。学生获得的是这棵知识树；我备课使用的是由同一批节点构成的有向图。

4

应用

根据自身需求搭建工具

1. 自己搭建的工具
2. 案例：教学平台
3. 平台的开发与上线
4. 示例：政策文件网站
5. 示例：用网页制作视频
6. Vibe Coding

应用：根据自身需求搭建的工具

本讲介绍的每一个工具，都源于一项没有现成产品可以满足的需求。这些工具都不需要我亲自编写代码，但都需要我把需求表述清楚。

提取练习脚本

约 200 行。每天练习五条，只显示标题；根据日志自动计算完成率。

站点校验器

在 1,629 份文档中检查数据加载、ID 与失效链接；这一关卡是生成的网站可信的前提。

lint 与反向链接脚本

逐条对照模板检查，并重新生成索引和图谱。一致性由工具保证，而非依靠自觉。

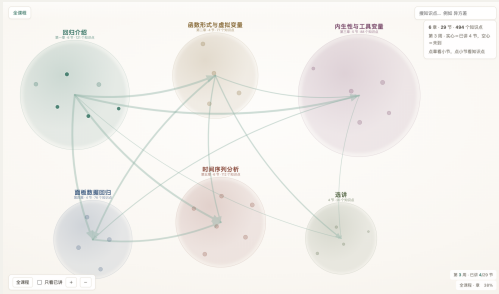
Research Planner

研究需等待审稿，按截止日期安排的日历并不适用。该工具按进度安排，并提示上次停在何处。

搭建过程均为相同的四个步骤：**说明需求、由 AI 实现、检查结果、提出下一步要求。**

主要案例：基于课程材料搭建的教学平台

第三部分提到的三门课程材料（讲义、课件、讲稿、课堂转写）还有另一项用途：教学平台 `taich.tech/teaching`，于 9 月 11 日上线，三门课程共用。



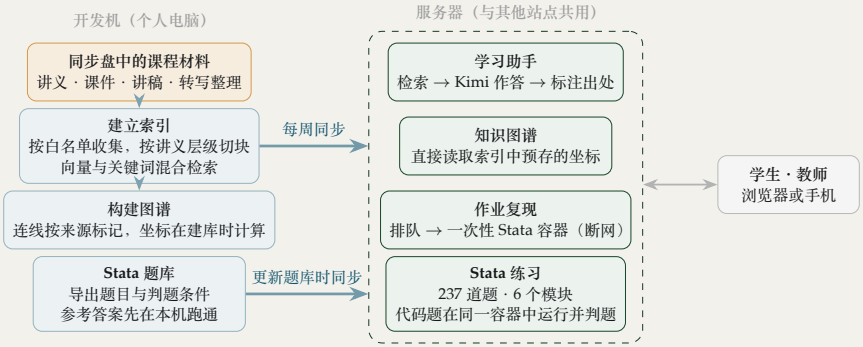
计量经济学的知识图谱：29 个小节、494 个知识点。

学习助手	仅依据本课讲义作答，每句注明出处
知识图谱	知识点、先修关系与授课周次
作业复现	在隔离容器中重新运行学生的 Stata 代码
Stata 练习	237 道题，代码题在容器中自动判题

规模与投入

三门课程共 2,429 块语料、1,170 个知识点。9 月 3 日首次提交代码，至 9 月 15 日首次作业上线，共 58 次提交，其中 42 次与 Claude Code 协作完成；Python 代码约 7,700 行。

平台架构：开发机建库，服务器读取



技术选型依据当前规模确定

不使用向量数据库：2,429 块语料全量检索只需一次矩阵乘法，耗时不到 10 毫秒 · 不使用 **Redis**：队列即数据库中的一张表 · 不使用前端打包：采用服务端渲染 · 每门课程使用独立索引：若共用索引，出处会在课程之间混淆

学习助手：相比通用 AI 增加的是专属信息

与通用 AI 相比，该助手增加的信息全部来自课程本身：



我

遗漏变量偏差的方向怎么判断？

助手

遗漏变量偏差的方向由两个符号共同决定：遗漏变量对被解释变量的影响（β_2）和遗漏变量与解释变量的相关方向（δ_1）。偏误方向就是 $\beta_2\delta_1$ 的符号 [3]。

具体判断规则如下表 [3]：

β_2：X_2 对 Y 的影响	δ_1：X_2 与 X_1 的相关方向	$\beta_2\delta_1$	偏误方向	结果
+	+	+	向上偏误	高估 β_1
+	−	−	向下偏误	低估 β_1
−	+	−	向下偏误	低估 β_1
−	−	+	向上偏误	高估 β_1

需要注意两点：

1. "向上"和"向下"是指估计量的期望相对于真实值 β_1 的位置，与系数的绝对值大小无关。当 $\beta_1 < 0$ 时，向上偏误表现为负效应的绝对值被缩小，甚至可能使估计出的系数变为正号 [3]。
2. 如果存在多个遗漏变量，偏误公式推广为 $E(\hat{\beta}_1) = \beta_1 + \beta_2\delta_{21} + \beta_3\delta_{31}$，各项可以叠加，也可以部分抵消。遗漏变量越多，方向判断越不可靠 [1][4]。

出处 · 点开看原文

- 讲义 [1] 1.6 › 遗漏变量偏误 › 多个遗漏变量
- 讲义 [3] 1.6 › 遗漏变量偏误 › 偏误方向的判断
- 课件 [4] 1.6 › 遗漏变量偏误 › 多个遗漏变量的情形

- **本课的讲法和符号**：以讲义为准，不以通用知识「纠正」教师的符号约定
- **每句注明出处**：点击即可查看讲义原文；补充内容注明「不考」
- **掌握授课进度**：16 周的进度表已解析并纳入索引
- **材料缺失时分级说明**：本课材料涵盖的，按本课作答；其他课程涵盖的，标明课程名称；三门课程均未涵盖的，开头即说明「课程材料里没有这一段」，并标注「材料外」

分级阈值依据实测确定：本课讲授过的问题，最高相似度为 0.70–0.76；三门课程均未讲授的，全部 ≤ 0.55。

知识库并非越全越好：各门课程都有易混淆的旧材料

若误收一份材料，出处便会指向另一套内容，**且系统不会报错**。三门课程均遇到了这一问题：

课程	冲突所在	处理方式
计量经济学	编号经过调整，旧编号的材料仍留在目录中	按路径排除旧目录
营销分析	2025 版与 2026 版并存：同为 1.1 节，一版讲 A/B 测试，另一版讲「为什么简单比较会失败」	仅收录 2026 版
商务数据分析	本课讲义与沿用自计量课程的讲义混杂，其中五份逐字节相同	仅收录标题与本课同节号课件一致的讲义
计量讲义	讲义包含习题与参考答案，占语料的 24%，其中 166 块为明确答案	按章节标题排除，并以测试确保残留为 0

第三部分所讲的准入门槛，在面向学生的系统中体现为**白名单**：默认不收录，收录范围需明确列出。

知识图谱：三类连线及其可信度

连线	来源	纳入图谱的方式
层级	章 → 节 → 知识点	直接获得，以位置表示，不画线
先修	进度表中的授课顺序；讲义中作者写明的「在 3.1 节会展开」；教师标注的先修关系	由作者本人给出，直接采用
相关	模型按节提出，每条附一句理由	通过关卡后才向学生展示

「相关」类连线需要设置关卡的原因

最初按向量相似度建立连线，约一半可用。合理的例子如「总体回归模型与术语」与「总体、样本与两组符号」(0.78)；不合理的例子如「这门课回答什么问题」与「选讲：受限因变量模型」(0.77)，两段均类似导语，仅文风相近。学生会将图谱视为课程的权威结构，**含一半噪声的图谱不如没有图谱**。

改为由模型按节提出、每条附一句理由后，质量明显提高，其中 47% 为跨章连线。

审核过严导致功能失效：关卡的调整



教师审阅页：每条候选附一句理由，可保留或删除；通过后才会出现在学生的图谱中。

第一版：教师逐条审核

三门课程共六百多条候选，实际仅审核了 3 条。学生打开知识点时，「要先理解 / 后面会用到」几乎全部显示「暂无」：计量课程 494 个知识点中只有 4 个带有关系。

当前：模型结合原文复核，教师可调整

结合两端的讲义原文，按「不先懂 A 就理解不了 B」的严格标准判断：665 条中通过 390 条。未通过的不删除，保留在审阅页；教师已审核的条目不作改动；每条的判断与理由单独保存，可追溯由谁批准通过。

作业复现：基于 84 份真实作业确定规则

学生提交 Stata do 文件，平台在一次性容器中重新运行并比对数值。开发之前，先对去年的 84 份真实提交进行了统计：

防范对象依据数据确定

`shell` 从未出现；三分之一的文件 `cd` 到学生本机的路径。问题不在于恶意代码，而在于路径。

第一版按命令黑名单拒绝，被拒绝的 4 份**均为误判**（导出描述统计表、删除自建的临时文件）。改为检查路径是否超出工作目录后，误判为 0，自动改写 176 处。

比对方式基于学生的已有习惯

59 份使用了 `esttab`，44 份使用了 `estimates store`。因此只需增加一行 `platexport`，将估计结果导出为 (模型, 变量, 统计量) 的长表，按相对容差比对。基准由平台在同一沙箱中运行标准答案生成。

实际运行中发现的问题：一个正则表达式导致 25 次报错

改写日志命令时只替换了文件名，学生原有的 `, replace text` 保留在后面，导致出现两次 `replace`。修复该问题及另一个隔离过度的 bug 后，通过率由 24% 提高至 36%；其余失败多为引用了未提交的中间文件或变量名不一致，因为去年是论文作业，本身就无法直接运行。

部署到国内服务器：上线前后遇到的五个问题

1. **依赖无法安装**：PyPI 和 Docker 镜像均需切换为国内源，否则构建会因超时而失败。
2. **索引不在代码仓库中**：数据目录不纳入 git，在服务器上 clone 后直接启动，首页显示「还没有可用的课程」。索引只能在开发机上建好后再同步到服务器。
3. **单个提问导致全站阻塞**：一次问答耗时 15–40 秒，异步函数中调用了同步网络请求，阻塞了整个事件循环。一名学生提问时，健康检查的响应时间由 1.5 毫秒增至 38 秒。
4. **耗时较长的请求被中断**：反向代理默认超时为 60 秒，遇到耗时较长的问答时，学生收到 504 错误，而后台实际已完成作答。
5. **配置文件被合并为一份**：同一个数据库路径在容器内与宿主机上并不相同。配置合并后，批改进程新建了一个空数据库并持续轮询；系统不报错，但作业始终无人批改。

以上问题均已写入部署文档；上线前另有自检脚本，检查索引、模型调用、预览后门与默认口令。

测试的可靠性：项目中三次虚假的「通过」

项目中的代码和测试大多由 AI 编写。其中有三次测试显示「通过」，事后发现并未起到检验作用：

1. **许可证信息屏蔽**：Stata 日志的开头印有软件序列号，而学生可以查看自己的日志。屏蔽代码按空格切分词语，无法识别单独出现的序列号；自测使用的字符串在真实日志中并不会出现，因此显示「通过」。改为使用与真实文件结构相同的样本测试，并确认旧代码在新测试下会失败。
2. **整组测试被跳过**：索引改为按课程分目录存放后，测试中一个硬编码的旧路径导致四组测试被整体跳过，但输出仍显示「全部通过」。
3. **并发 bug 的测试**：上线当天，两个进程同时为数据库添加列，其中一个进程崩溃。前两版测试的结果均不可靠：一版中两个进程并未真正发生冲突，另一版虽然失败，但失败原因与线上问题不同。直到用有缺陷的版本复现出线上的报错信息，测试才算有效。

原则

一条测试应先确认其在有缺陷的代码上失败，才能相信它在正确代码上的通过结果。

上线之后：真实使用修正了小样本得出的结论

23 个真实提问中 12 个被判为「材料里没有」

9 月 12 日，商务数据分析课程的两位同学开始连续追问。排查后发现两个原因：选择「全部课程」后，检索范围却始终停留在开头一节；「∂ 是啥」「帮我再详细解释一下」这类简短追问不含主题词，达不到检索阈值，而上一轮刚引用的讲义恰好涵盖这一内容。改为追问检索不到结果时沿用上一轮的出处，并用真实模型重放了这两段对话。

基于三道题的结论被六道题的测试推翻

在三道简单题上，关闭「深度思考」后答案依然正确，输出 token 减少六成，因此最初默认关闭。换用六道需要推理的题目后，关闭思考的模型在一道题上给出了确定但错误的回答，计算过程与结论相互矛盾。对教学助手而言，这是最严重的一类错误，因此改为由学生自行选择是否开启。

教师端获得的信息

学生的提问按小节汇总，真正的难点由此显现，远早于期末考试；「材料里没有」一栏是讲义缺口最直接的信号。专属材料因此每周都在不断充实。

示例：将一批政策文件整理为网站

研究问题：哪些政府与高校政策涉及高等教育中的 AI？政策脉络如何，高校层面如何推进？

这是第三部分所述报告证据库的副产品，已整理为可检索的站点。

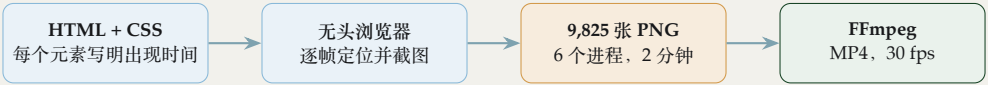
提示词分为三个步骤

1. 收集：原文及标题、日期、发文字号、URL，共 **1,629** 份文件
2. 分析：为 **61** 份文件建立全文语义记录，包括概念、方向、层级
3. 搭建与校验：搭建包含六个标签页的站点，再用 `validate_site.mjs` 检查数据加载、ID 与失效链接

六个标签页：总览 · 顶层逻辑 · 高校实践 · 工作流经验 · 语义分析 · 全量证据库

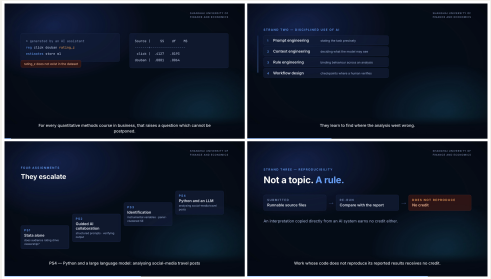
以网页方式制作的五分钟视频

一支课程宣传片：时长 5 分 27 秒、1080p，旁白使用我本人的声音，语音模型仅依据 20 秒录音生成。制作过程不使用剪辑软件，无需注册账号，也无需上传。修改一句旁白后，约四分钟即可生成新版本。



- 版面采用普通的 HTML 和 CSS；每一帧单独计算，因此渲染可以并行且可复现，不依赖时钟和随机数，也不使用 CSS 动画
- **画面时间由声音决定**：旁白切分为 45 句，逐句生成并计时；再由脚本将实测时长写回 HTML，使字幕与配音精确同步，修改一句后，后续内容自动顺延

成片效果与技术构成



成片中的四帧。

声音：基于 20 秒录音克隆

Chatterbox，MIT 许可，在笔记本电脑上运行。学术旁白需要调整两处：降低表现力，语速设为约每分钟 **165** 词；默认的 216 词接近广告语速。对外发布时须标注 AI 合成。

技术栈：三个开源组件

HyperFrames 负责渲染，Chatterbox 负责配音，FFmpeg 负责合成。432 行的生成器由 AI 编写；我负责撰写旁白和检查。

适用情形

视频日后需要修改、内容本身结构清晰，或素材不能离开本机。

自动检查的局限：视频制作中的经验

HyperFrames 的 `check` 包含五项检查（语法、可定位、重叠、可复现、对比度），在 9,825 帧中查出 3 处时间冲突、10 处对比度不足、2 处文字重叠。但成片中最严重的两个错误，它均判定为通过。

四张卡片在两秒内全部出现

旁白逐句介绍四项作业，卡片却按固定的 0.72 秒间隔出现，导致画面已经展示完毕，旁白尚未开始介绍。修改方法：将每个元素绑定到对应语句的起始时间，时间从时间表文件中读取。

一张照片遮挡了字幕

结尾处一张全屏课堂照片覆盖了字幕、校名和进度条。这在技术上完全合规，检查器无法发现。修改方法：为常驻图层固定堆叠顺序。

由此得出的原则

`check` 通过之后，每个场景至少抽取一帧进行人工查看。技术检查只能发现技术错误，视觉上的问题需要人工查看才能发现。

检查必须独立于被检查的对象

同一教训在更一般层面上的体现，来自同一个月内的另一件事。

与盲区重合的完整性检查

我用正则表达式从 PDF 中解析一份班级名单，脚本遗漏了一整页；而完整性检查报告无遗漏，因为它基于同一套正则表达式编写，无法发现这套正则遗漏的内容。最终发现问题的，是名单本身的最后一行：「共 117 名学生」。

- 字幕应对照**实测时长**检查，而非对照生成字幕的脚本
- 画面应对照**抽取的帧**检查，而非对照检查器的报告
- 台账应对照**文件**核对，论断应对照**定位到页码的原件**核对（第三部分），关键事实需人工核实
- 用 AI 构造变量时的抽检也遵循同一原则：人工标注的样本应独立于为其余样本打标签的模型

Vibe Coding：四个环节与适用边界

Karpathy 于 2025 年 2 月提出这一术语，柯林斯词典将其评为年度词汇。2026 年，Karpathy 将其专业版本改称为 *agentic engineering*（「vibe coding 抬高下限，agentic engineering 抬高上限」）；核心循环仍是四个环节。

循环（由使用者负责验证）

1. 用平实的语言**说明需求**
2. **AI 编写代码**
3. **检查结果**：页面与功能是否正确
4. **提出下一步要求**

适合：产出可以直接查看验证的项目

个人研究网站；数据看板；简单的内部工具；原型；自动化脚本（清洗、批处理）

需谨慎：错误不易被发现的项目

存放敏感数据的系统（需要审计）；上万行的代码库（需要架构）；交易类应用（需要严格测试）；多人大型项目（需要代码评审）

研究者常用的网站、可视化与脚本大多属于第一类。相比编写代码的能力，**说明需求**和**判断结果**更为重要。

条目长期保存，结构按需重建

Robert Greene

两千张卡片保存了几十年；撰写《权力的 48 条法则》时，按该书的 48 章重新分类。

John McPhee

每篇文章重新建立完整结构，完成后即拆散；笔记也用剪刀剪开，分入该篇文章的文件夹。

- 两位高产作者在条目上的做法相反（一位永久保存，一位物理销毁），在结构上却完全一致：**结构为单次产出而编排，完成后即可拆除**
- 这可以解释第三部分的多项数据：互链稀疏（0.6）并非懈怠，无人走完成熟度阶梯也不代表失败；**精力主要投入在条目上，结构本身就是临时的**
- 撰写论文、备课、准备报告时，从条目中**临时编排结构**，用完即拆除，条目放回原处

依据两份接近一手的材料（研究助理与本人自述）· 与本讲 82 张卡片案例的做法相互独立而吻合。

可以着手尝试的三件事

1. **将重复三次以上的工作整理为 Skill**
   先编写 SKILL.md 触发层的三行：名称、用途与触发条件。准备 3–5 篇语料即可完成整个流程。

2. **为一个实际项目编写 WORKFLOW.md**
   只写三条规则和一道 Gate。规则应具体到可以判定通过或退回，避免「代码要规范」这类表述。

3. **建立一个文件夹，并编写约三十行的入口文件**
   原料与成品分开存放；此后每条笔记开头写一句可被反驳的核心命题，已有笔记无需补写。

Q & A 谢谢！

陶学臻 · 上海财经大学商学院

tao.xuezhen@mail.sufe.edu.cn

微信公众号

AI 经研派

扫码关注

本讲资源网站

taich.tech

资源包 · 讲义 PDF

教学示例网站

SUFE-MKT-analysis.online

营销分析教学网站